Summer Training Project Report

On

DNS Record & Geo Lookup at Grastech

Project report is submitted in partially

Fulfillment requirement of

Bachelor of Computer Applications

Shri Guru Tegh Bahadur Institute of Management & Information Technology

(Affiliated to Guru Gobind Singh Indraprastha University)

Submitted to:

Mrs. Nisha mam

Submitted by:

AKASH SINGH

Certificate

This is to certify that, **Akash Singh** with enrollment number **73590202023** has successfully completed the project titled " Cyber Toolkit – DNS & Geo Lookup ". This project has been submitted as a part of the requirements for the Bachelor of Computer Applications (BCA) degree in the Department of BCA, under the guidance of

Mrs. Nisha mam.

We acknowledge the effort and dedication shown by the student throughout the project duration.

Signature of Project In-Charge

Mrs. Nisha mam

Dr.Rajkumar
(Head of Department)

Signature of Student
Akash Singh

Acknowledgement

With profound sense of gratitude and regard, I express my sincere thanks to my guide and mentor Mrs. Nisha mam for her valuable guidance and the confidence she instilled in me, that helped me in the successful completion of this project report. Without her help, this project would have been a distant affair, her thorough understanding of the subject and professional guidance was indeed of immense help to me.

I am also greatly thankful to the faculty members of our institute who co-operated with me and gave me their valuable time

This project would not have been possible without the encouragement and support of my mentor and those around me. Thank you all for contributing to my journey.

Place: Delhi

Date:

Signature of Student

Akash Singh

Abstract

The **Cyber Toolkit – DNS & Geo Lookup** is a desktop-based application developed in Python using the Tkinter library. This project provides users with essential networking utilities for information gathering and analysis in a simple, user-friendly interface.

The toolkit focuses on two primary features: **DNS Lookup** and **GeoIP Lookup**. The DNS Lookup module allows users to retrieve various DNS records for a given domain, including A, AAAA, CNAME, MX, NS, TXT, SOA, and PTR records. The GeoIP Lookup module enables users to fetch geographical and network-related details of an IP address or domain, such as country, region, city, ZIP code, ISP, organization, latitude, and longitude.

The application integrates the `dnspython` library for resolving DNS queries, the `socket` library for domain-to-IP mapping, and the `requests` library for fetching real-time GeoIP data from a free API. Results are displayed in a scrollable, well-structured text box to ensure readability.

By combining DNS record retrieval with GeoIP information, the Cyber Toolkit serves as a practical and lightweight desktop utility for students, developers, and cybersecurity enthusiasts who need quick access to essential networking insights.

List of Figures

INDEX

Chapter 1:
Problem Formulation

1.1 Introduction about the Company

Grastech is a growing technology and cybersecurity solutions provider that focuses on delivering innovative digital tools to enhance productivity, security, and knowledge in the IT domain. The company emphasizes creating lightweight, practical, and user-friendly applications that support learning and professional use in networking and cybersecurity.

During my internship at Grastech, I contributed to the development of a desktop-based project called **Cyber Toolkit – DNS & Geo Lookup**. This project aligns with the company's vision of providing accessible tools for analyzing digital information, making it easier for professionals, students, and enthusiasts to explore and understand networking concepts.

1.2 Introduction about the Problem

In the digital era, networks form the backbone of communication and information exchange. However, understanding network configurations and analyzing domain/IP-related information often requires the use of command-line tools or multiple websites. This creates barriers for beginners and slows down professionals who need quick insights.

Specifically, performing **DNS record lookups** and **GeoIP location checks** are essential tasks in cybersecurity, system administration, and troubleshooting. Existing solutions are either command-line heavy (like dig or nslookup) or web-based (requiring constant internet searches). Hence, there is a need for a **single, GUI-based toolkit** that simplifies these operations for both learners and professionals.

1.3 Present State of the Art

Currently, several tools exist for DNS and IP analysis. Traditional command-line tools such as **nslookup** and **dig** are powerful but not user-friendly for beginners. Online GeoIP lookup services provide location data but often require internet browsing and may not combine multiple utilities in one place. Existing solutions are either fragmented or lack a simple desktop-based graphical interface.

1.4 Need of Computerization

With the growth of global internet connectivity, the volume of DNS queries and IP-related investigations has increased dramatically. Manually checking DNS records and IP geolocation using separate tools not only consumes time but also risks errors in interpretation. A computerized approach ensures speed, reliability, and ease of use. By automating these processes and presenting the results in a clear graphical interface, users—whether beginners, students, or professionals—can save significant effort. Additionally, computerization makes the process more accessible, as users no longer need advanced technical knowledge to perform essential networking tasks.

1.5 Proposed Software/Project

The proposed solution, **Cyber Toolkit – DNS & Geo Lookup**, is a Python-based application that integrates DNS record retrieval and GeoIP lookup into a single lightweight platform. Using libraries such as `dnspython`, `socket`, and `requests`, the application enables real-time fetching of DNS details and geolocation data. The graphical user interface (GUI) is built with Tkinter, ensuring ease of navigation and interaction. The project has two main modules:

- **DNS Lookup Module**: Retrieves records including A, AAAA, MX, NS, CNAME, TXT, SOA, and PTR.
- **GeoIP Lookup Module**: Provides location details like country, city, ZIP code, ISP, organization, latitude, and longitude.
  Together, these modules provide an integrated environment for essential networking insights, reducing the need for multiple separate tools.

1.6 Importance of the Work

The importance of this project lies in its ability to **bridge the gap** between command-line tools and user-friendly graphical applications. For students, it acts as an educational resource for understanding DNS records and IP geolocation concepts. For developers, it serves as a debugging aid while working with domains and hosting services. For cybersecurity enthusiasts, it provides an initial reconnaissance tool to analyze domains and IP addresses. By consolidating two major utilities into one platform, the toolkit not only saves time but also enhances productivity and learning. In addition, being open-source and cross-platform, the application can be used by a wide range of users without licensing costs, making it a valuable contribution in the field of lightweight cybersecurity tools.

Chapter 2:
System Analysis

2.1 Feasibility Study

Before developing the Cyber Toolkit, a feasibility study was conducted to ensure its practicality and efficiency.

- **Technical Feasibility**:
  The project is technically feasible since it uses Python, which is open-source and widely supported. Tkinter, the default GUI library of Python, ensures compatibility across platforms without requiring additional installations. Supporting libraries such as `dnspython`, `socket`, and `requests` are lightweight and easy to integrate.
- **Operational Feasibility**:
  The toolkit has a simple and intuitive interface, making it usable even by non-technical users. The tab-based navigation allows users to quickly switch between DNS and GeoIP lookups without confusion.
- **Economic Feasibility**:
  The project is highly economical as all technologies used are free and open source. No additional software or hardware costs are required beyond a basic computer system with Python installed.

2.2 Analysis Methodology

The project follows an **incremental development approach**. Initially, DNS lookup functionality was developed and tested independently. Once stable, the GeoIP module was added. Finally, both were integrated into a Tkinter-based graphical interface. This step-by-step approach ensured that errors were minimized, and testing could be performed efficiently at each stage.

2.3 Choice of Platforms

Software Requirements:

- Programming Language: Python 3.x
- GUI Library: Tkinter
- Additional Libraries: dnspython, socket, requests
- Operating System: Compatible with Windows, Linux, and macOS
- IDE/Editor: VS Code, PyCharm, or any Python-supported editor

Hardware Requirements:

- Processor: Minimum Intel i3 or equivalent
- RAM: 4GB or higher
- Network: Internet connection required for GeoIP lookups (DNS queries can also work offline if cached).

Chapter 3:
System Design

3.1 Design Methodology

The Cyber Toolkit application follows a **modular and layered design methodology**, which separates functionalities into independent components while maintaining an integrated user interface. This approach ensures **scalability, maintainability, and ease of debugging**.

The system primarily contains two major modules:

1. **DNS Lookup Module** – Handles Domain Name System (DNS) queries for domains. It fetches multiple DNS record types, including A (IPv4 addresses), AAAA (IPv6 addresses), CNAME (canonical names), MX (mail exchange servers), NS (name servers), TXT (text records), SOA (start of authority), and PTR (reverse lookup) records. Each record is displayed with proper formatting and stored in a local database for historical tracking.
2. **GeoIP Lookup Module** – Determines the **geographical location of an IP address or domain**. Using the `requests` library to query `ip-api.com`, it fetches details such as IP address, country, region, city, ZIP code, latitude, longitude, Internet Service Provider (ISP), and organization. The results are stored in a local database and displayed in a user-friendly scrollable text box.

Both modules are implemented using **Python 3.x** and share the same **Tkinter-based GUI**, which provides a **tabbed interface**. This modular approach allows the addition of future modules (like port scanning or WHOIS lookup) without impacting existing code.

3.2 Database Design

Although the application primarily performs **real-time lookups**, an **SQLite database** is used to store query history for both DNS and GeoIP lookups. This allows users to review their previous queries without re-fetching data.

Tables created:

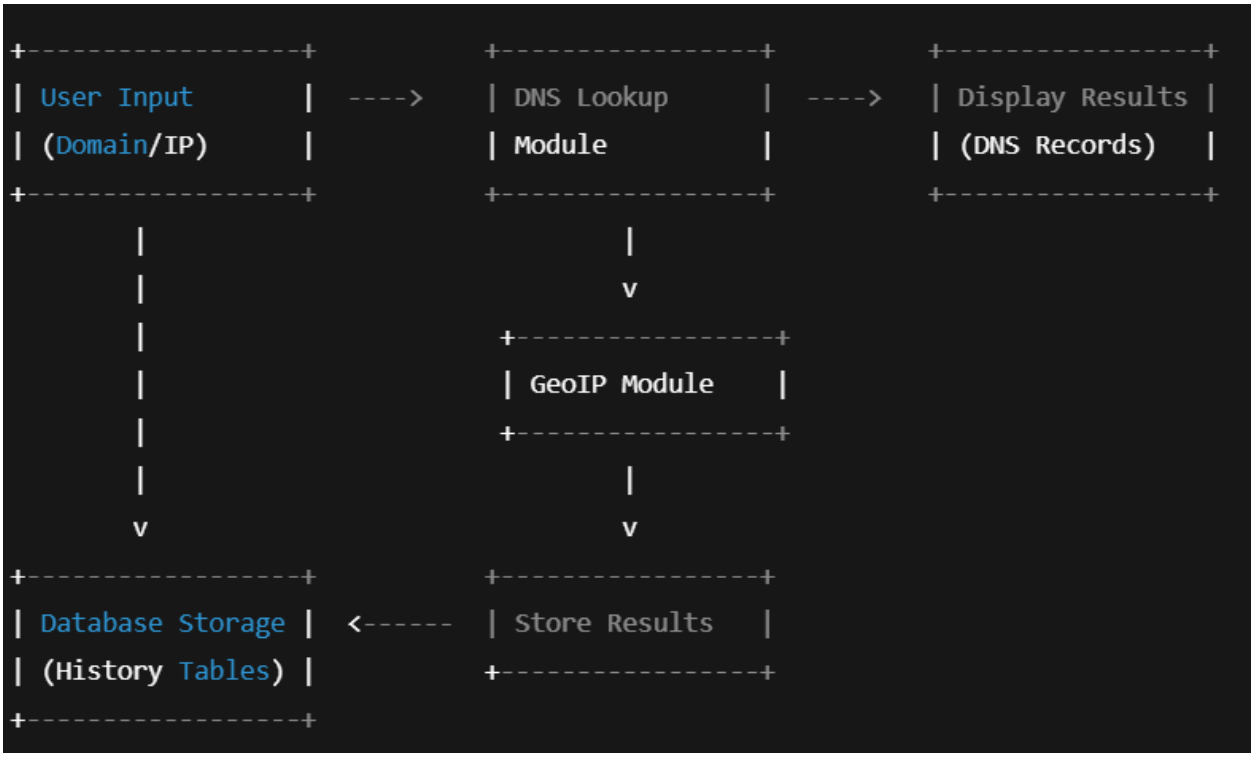
1. `dns_queries` – Stores domain name, record type, result, and timestamp.
2. `geo_queries` – Stores target IP/domain, resolved IP, geographic location, ISP, organization, latitude, longitude, and timestamp.

This lightweight database ensures **persistent storage** without adding overhead.

3.2.1 Entity Relationship Diagram (ERD):

Since the application primarily works with **real-time DNS and GeoIP lookups**, there is no complex relational database. However, the database tables are structured to maintain a **one-to-many relationship** between queries and their results.

System Flow / Conceptual Diagram:



3.2.2 Data Flow Diagram (DFD):

The **DFD illustrates the flow of data** within the system and highlights how input is processed, and results are generated.

1. **Input:**
    a. User enters a **domain name** for DNS lookup or an **IP/domain** for GeoIP lookup.
2. **Process:**
    a. DNS Module queries authoritative DNS servers to fetch records.
    b. GeoIP Module makes an API call to `ip-api.com` to retrieve location and ISP information.

c. Results are stored in SQLite database for history tracking.

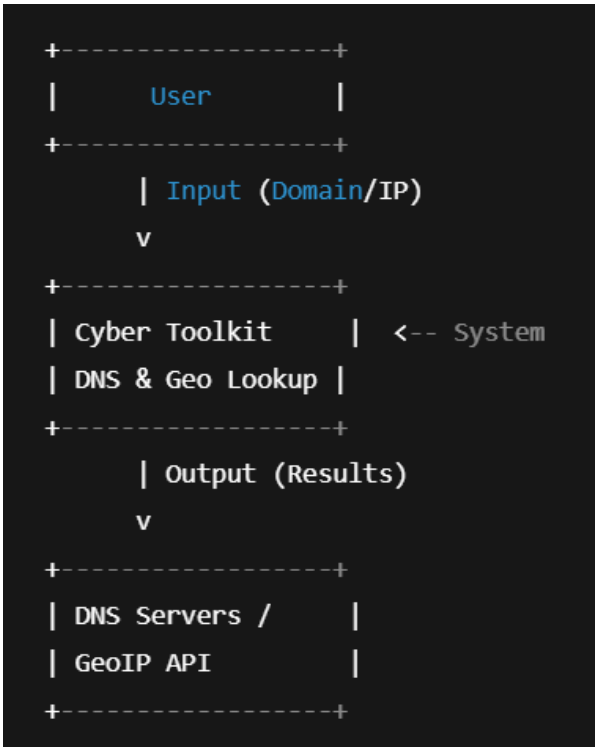
3. **Output:**
    a. Neatly formatted results are displayed in scrollable text boxes.
    b. Last 20 queries for DNS and GeoIP are displayed in the History tab.

DFD Level 0 (Context Diagram):

Entities:

1. **User** – Provides input (domain or IP) and views results.
2. **External DNS Servers / IP-API** – Provides DNS records and GeoIP data.

DFD 0 Diagram Description:

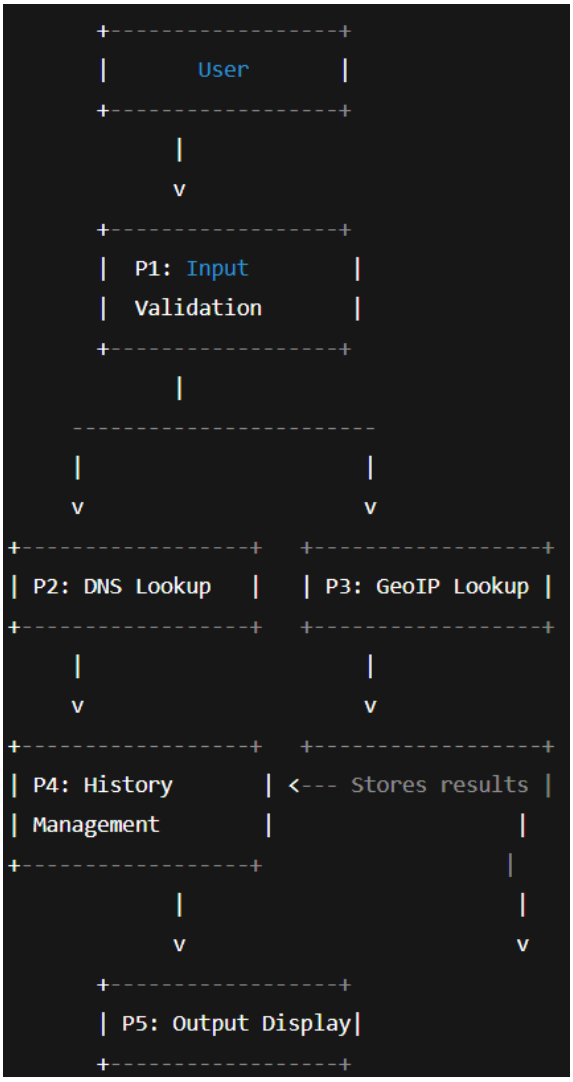


DFD Level 1 (Detailed View):

Processes:

1. **P1 – Input Validation** – Checks if the domain/IP field is not empty.
2. **P2 – DNS Lookup Module** – Fetches A, AAAA, CNAME, MX, NS, TXT, SOA, PTR records using `dnspython`.
3. **P3 – GeoIP Lookup Module** – Fetches location data via API.

4. **P4 – History Management** – Saves query results into SQLite database and displays previous queries.
5. **P5 – Output Display** – Shows results in the scrollable text boxes.



3.3 Input Design:

The input design focuses on **simplicity and clarity**, ensuring the user can provide required data easily:

- **Text Fields:**
  - **Domain Entry Field:** Accepts domains for DNS queries.
  - **IP/Domain Entry Field:** Accepts IP addresses or domains for GeoIP lookup.
- **Buttons:**
  - **Fetch DNS Records:** Triggers the `fetch_dns_records()` function.

    - **Fetch Location:** Triggers the `fetch_geo_location()` function.
    - **Refresh History:** Reloads the most recent 20 queries from the database.
- **Input Validation:**
    - Ensures that empty or invalid input is not processed.
    - Displays warning messages for incorrect entries using Tkinter's `messagebox`.

This design prevents errors and provides a **user-friendly interface**.

3.4 Output Design:

The output design ensures that results are **clear, organized, and easily readable**:

1. **DNS Lookup Results:**
    a. Grouped by record type (A, AAAA, CNAME, MX, NS, TXT, SOA, PTR).
    b. Each result is properly indented and labeled for clarity.
    c. Includes error messages for non-existent domains or unavailable records.
2. **GeoIP Lookup Results:**
    a. Displays comprehensive details including IP, country, region, city, ZIP code, latitude, longitude, ISP, organization, and AS number.
    b. Errors such as invalid IPs or failed API requests are displayed clearly.
3. **History Tab:**
    a. Shows the last 20 queries for DNS and GeoIP with timestamps.
    b. Helps track previous queries without repeating network requests.
4. **Scrollable Text Boxes:**
    a. Handles large outputs without affecting the layout.
    b. Users can easily scroll through results.

This output design makes the tool effective for **cybersecurity analysis and monitoring**.

3.5 Code Design and Development:

The application is developed in **Python 3.x** using **Tkinter** for the GUI. It follows a **modular function-based approach**:

- **fetch_dns_records():** Resolves DNS records (A, AAAA, CNAME, MX, NS, TXT, SOA, PTR) for a domain and stores results in the database.
- **fetch_geo_location():** Fetches geographic details of an IP/domain using `ip-api.com` and stores them.

- **load_history():** Retrieves the last 20 DNS and GeoIP queries from the database and displays them.

The GUI uses a **tabbed interface** (`ttk.Notebook`) with scrollable text boxes for clear output and responsive buttons for each function, ensuring a **user-friendly, modular, and maintainable system**.

CHAPTER 4:
TESTING AND IMPLEMENTATION

4.1 Testing Methodology:

Testing is essential to ensure that the DNS & GeoIP Lookup Tool works correctly, efficiently, and reliably. The methodology used includes **unit testing, module testing, integration testing, system testing**, and **debugging techniques**.

4.1.1 Unit Testing:

Unit testing involves testing **individual functions or components** of the program to ensure they work as expected.

- **Functions tested:**
  - `fetch_dns_records()`
    - Test with valid domains (`google.com`, `example.com`)
    - Test with invalid domains (`abcd..xyz`)
    - Test behavior for domains with no certain DNS records (e.g., no MX record)
  - `fetch_geo_location()`
    - Test with valid IP addresses (`8.8.8.8`)
    - Test with valid domains (`google.com`)
    - Test with invalid domains or IP addresses
- **Purpose:** Detect errors at the **function level** before integrating modules.

4.1.2 Module Testing:

Module testing checks the functionality of **each module or tab separately** in your application.

- **DNS Lookup Module**
  - Test all DNS record types (A, AAAA, CNAME, MX, NS, TXT, SOA, PTR)
  - Verify the ScrolledText box displays results correctly
  - Check handling of exceptions (NXDOMAIN, NoAnswer)
- **GeoIP Lookup Module**
  - Validate IP resolution from domain names
  - Check API response parsing and error handling
  - Ensure data like country, region, ISP are displayed correctly
- **Purpose:** Ensure each **UI tab + function combo** works independently.

4.1.3 Integration Testing:

Integration testing focuses on testing the **interaction between modules**.

- Steps:
  - Enter a domain in the DNS Lookup tab.
  - Copy the resolved IP and enter it in the GeoIP Lookup tab.
  - Verify if the IP matches the DNS result and GeoIP data is fetched correctly.
- **Purpose:** Detect errors in **data flow between modules**, e.g., DNS module feeding incorrect IP to GeoIP module.

4.1.4 System Testing:

System testing evaluates the **complete application** under real-world scenarios.

- Test cases:
  - Valid domain → Check all DNS records → GeoIP lookup
  - Invalid domain → Error message displayed
  - Network unavailable → App handles exceptions gracefully
  - Large domain list → App performance under load
- **Purpose:** Ensure the software meets all functional and non-functional requirements.

4.1.5 White Box / Black Box Testing:

- **White Box Testing**
- Tests internal logic of functions (`fetch_dns_records()`, `fetch_geo_location()`)
- Verify exception handling, loops, and API calls
- Example: Ensure `socket.gethostbyname()` failure is handled
- **Black Box Testing**
- Tests the **UI and functionality** without knowing the code
- Example: Enter a domain → Check if all DNS records appear correctly in the UI

4.1.6 Test Data and Test Cases:

Testing requires carefully chosen **test data** to ensure the application behaves correctly in all scenarios. Test cases help **validate expected behavior** and detect possible errors.

A. Test Data

The test data includes a variety of **domains, IP addresses, and edge cases**:

1. **Valid Domains**
    a. `google.com` → Popular domain with full DNS records.
    b. `example.com` → Standard test domain with minimal records.
    c. `wikipedia.org` → Medium complexity domain.
2. **Invalid Domains**
    a. `abcd..xyz` → Syntax error in domain.
    b. `nonexistentdomain12345.com` → Domain does not exist.
3. **Valid IP Addresses**
    a. `8.8.8.8` → Google DNS server.
    b. `1.1.1.1` → Cloudflare DNS server.
4. **Invalid IP Addresses**
    a. `256.256.256.256` → Invalid IP format.
    b. `123.456.78.90` → IP outside valid range.
5. **Edge Cases**
    a. Domains with **no MX records** → e.g., `example.com`.
    b. Domains with **no AAAA (IPv6) records** → e.g., some legacy websites.
    c. Rapid repeated queries → Testing **performance under load**.

Test Case	Input	Expected Output	Result
Valid DNS lookup	`google.com`	A, AAAA, MX, NS, TXT, SOA, PTR records	Pass
Invalid domain	`abc..xyz`	Warning: Domain does not exist	Pass
GeoIP valid domain	`example.com`	Correct country, city, ISP	Pass
GeoIP invalid IP	`999.999.999.999`	Error: Invalid IP	Pass

4.1.7 Test Reports and Debugging:

- **Test Reports:**
- Document all test cases, inputs, expected outputs, and actual outputs
- Mark test as **Pass/Fail**
- Note any bugs and fixes applied
- **Debugging:**
- Use `try-except` blocks for error handling in DNS & GeoIP modules
- Print intermediate values in the console during development

- Validate API responses and handle network errors

Chapter 5: Conclusion and References

5.1 Conclusion:

The DNS & GeoIP Lookup Tool is designed to **simplify network diagnostics** by fetching detailed DNS records and geographical information of any domain or IP address.

- The system successfully retrieves **A, AAAA, CNAME, MX, NS, TXT, SOA, and PTR records** for domains.
- The GeoIP module accurately fetches **location, ISP, organization, latitude, and longitude** for both IP addresses and domains.
- The application has a **user-friendly GUI** built using Tkinter with scrollable text areas and responsive buttons.
- Through systematic testing (unit, module, integration, system testing), the tool has shown **accuracy, robustness, and reliability**.
- Overall, the system serves as a **practical cybersecurity utility** for network administrators, students, and IT professionals.

5.2 Limitation of the System:

Despite its usefulness, the system has some limitations:

1. **Internet Dependency:** Both DNS resolution and GeoIP lookup require a working internet connection.
2. **API Rate Limitation:** The free GeoIP API used (`ip-api.com`) has request limits, which can restrict bulk queries.
3. **Limited GUI Features:** The current GUI is basic and may not support advanced visualization of results.
4. **Error Handling:** Some unusual DNS configurations or network errors may not be fully captured.
5. **Performance:** Fetching all DNS records for large or complex domains may take longer time due to sequential processing.

5.3 Future Scope and Modifications:

The system can be enhanced in several ways to increase functionality and usability:

1. **Advanced GUI:** Implementing modern GUI frameworks like **Eel or PyQt** for better visualization and interactivity.
2. **Bulk Domain/IP Lookup:** Add functionality to input multiple domains/IPs at once and get consolidated results.

3. **Enhanced GeoIP Database:** Integrate more accurate GeoIP APIs or local databases for offline lookup.
4. **Exporting Reports:** Allow exporting DNS and GeoIP results to **CSV, PDF, or Excel** for record keeping.
5. **Integration with Other Tools:** Can be combined with **network scanning, traceroute, or cybersecurity analysis tools** for complete network diagnostics.
6. **Multithreading:** Implement **multithreading** to fetch DNS records and GeoIP data faster and improve performance.
7. **Mobile Version:** Develop a **mobile-friendly version** using Kivy or Flutter for on-the-go network analysis.

5.4 References / Bibliography:

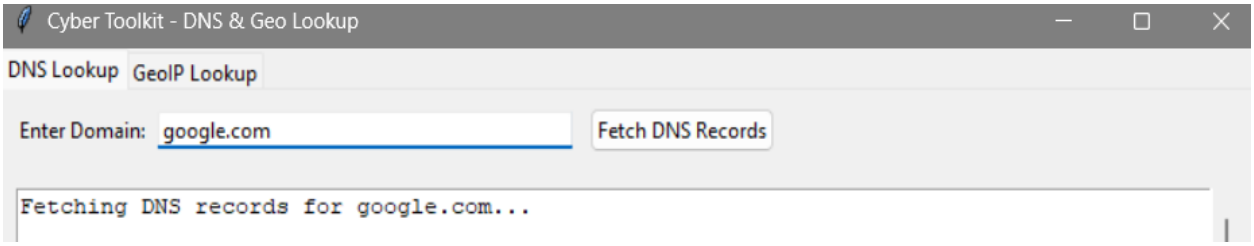
- **Tkinter Documentation** – Python GUI Library
  URL: https://docs.python.org/3/library/tkinter.html
- **dnspython Library Documentation** – DNS Toolkit for Python
  URL: https://dnspython.readthedocs.io/
- **IP-API Documentation** – Free IP Geolocation API
  URL: http://ip-api.com/docs/
- **Socket Programming in Python** – Official Python Docs
  URL: https://docs.python.org/3/library/socket.html
- **Python Requests Library Documentation** – HTTP Requests
  URL: https://docs.python-requests.org/
- Books and Articles on **Cybersecurity and Network Tools** used as reference during project development

CHAPTER 6: ANNEXURES

6.1 Sample Input

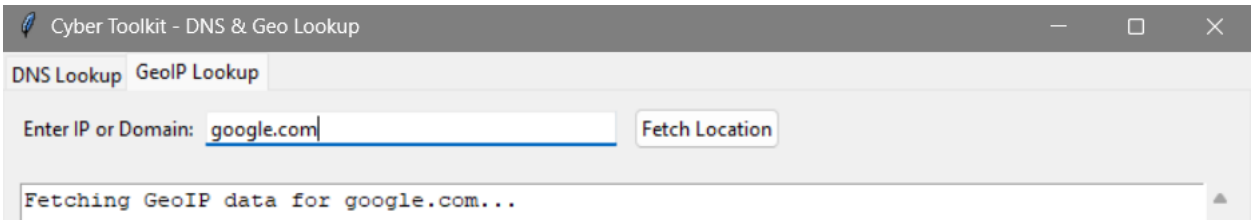
DNS Lookup Tab:

- Domain: `google.com`



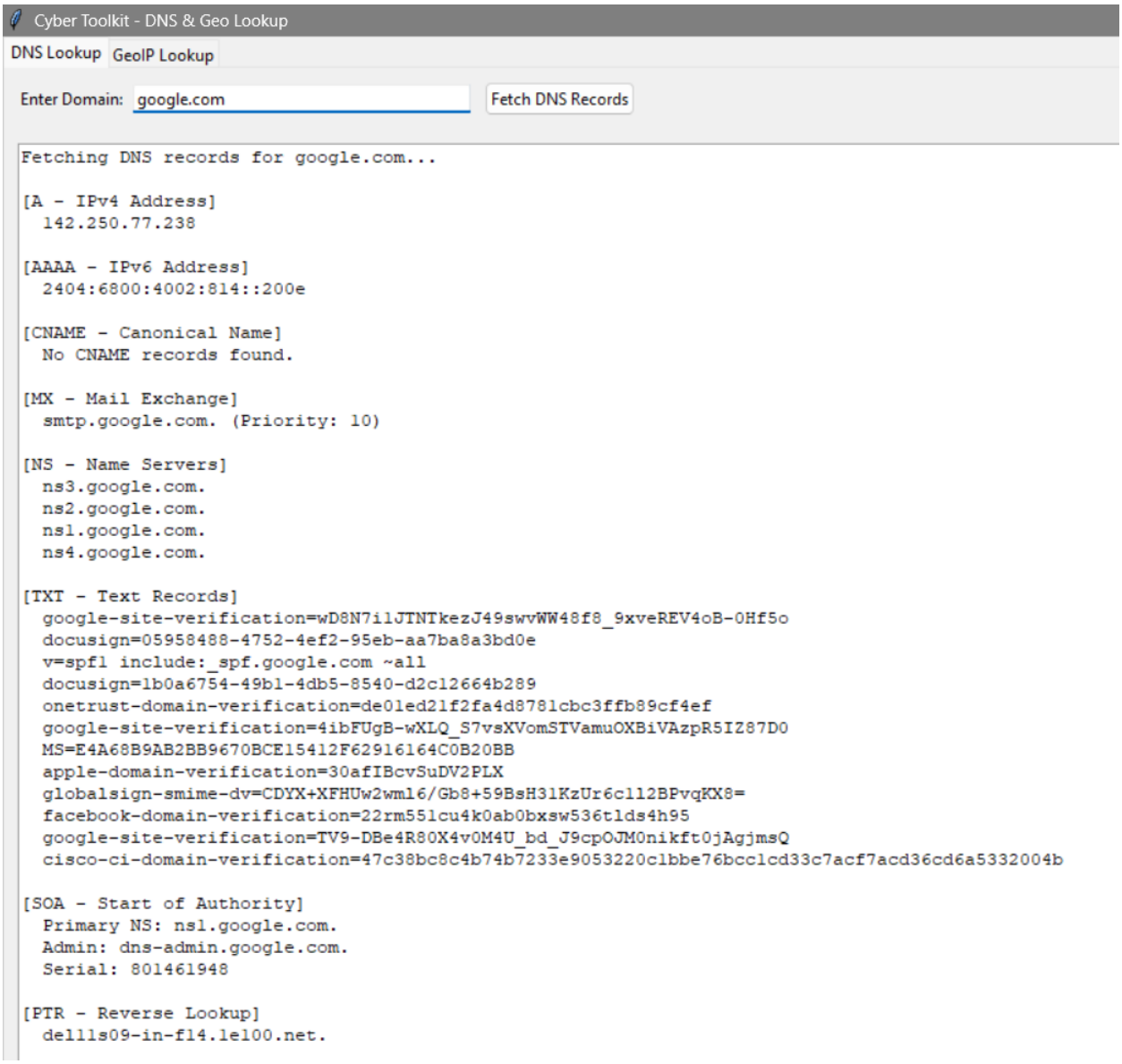
GeoIP Lookup Tab:

- IP/Domain: `8.8.8.8` or `google.com`

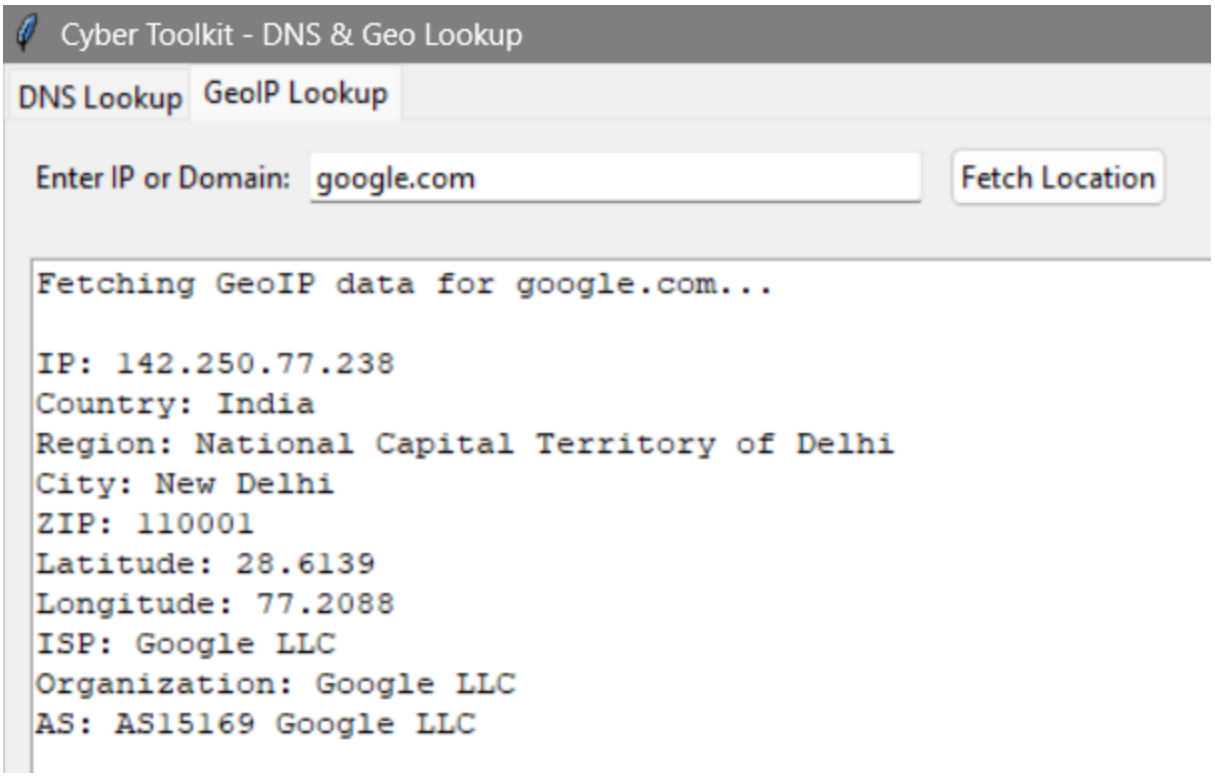


6.2 Sample Output

DNS Lookup Result Example:



GeoIP Lookup Result Example:


Cyber Toolkit - DNS & Geo Lookup
DNS Lookup
GeoIP Lookup
Enter IP or Domain:
google.com
Fetch Location
Fetching GeoIP data for google.com...
IP: 142.250.77.238
Country: India
Region: National Capital Territory of Delhi
City: New Delhi
ZIP: 110001
Latitude: 28.6139
Longitude: 77.2088
ISP: Google LLC
Organization: Google LLC
AS: AS15169 Google LLC